11. Trees

“Not all data grows in a straight line.”

Previous Section:

10. Queues

Contents:

So far, we have explored several fundamental data structures such as Arrays, Linked Lists, Stacks, and Queues. One thing these structures have in common is that they are **linear data structures**.

In a linear structure, elements are arranged in a sequence. Data is typically accessed in a straight path:

- Arrays are traversed from one index to another.
- Linked Lists move from one node to the next.
- Stacks operate from top to bottom.
- Queues process elements from front to rear.

Everything follows a predictable, sequential direction. But not all information in the real world is organized this way. Consider a company organizational chart, a family genealogy, or the folders on your computer. These structures are naturally hierarchical rather than sequential. To represent such relationships efficiently, we need a different approach.

This brings us to **non-linear data structures**. Unlike linear structures, non-linear structures allow data to branch into multiple paths. Elements may have hierarchical or interconnected relationships, enabling more flexible and efficient ways of organizing and accessing information.

One of the most important non-linear data structures is the **Tree**. Trees form the foundation of many advanced data structures and algorithms. Understanding them is a crucial step toward studying topics such as searching algorithms, databases, file systems, artificial intelligence, and network routing.

1. What is a Tree?

When most people hear the word *tree*, they imagine something with roots, a trunk, and branches.

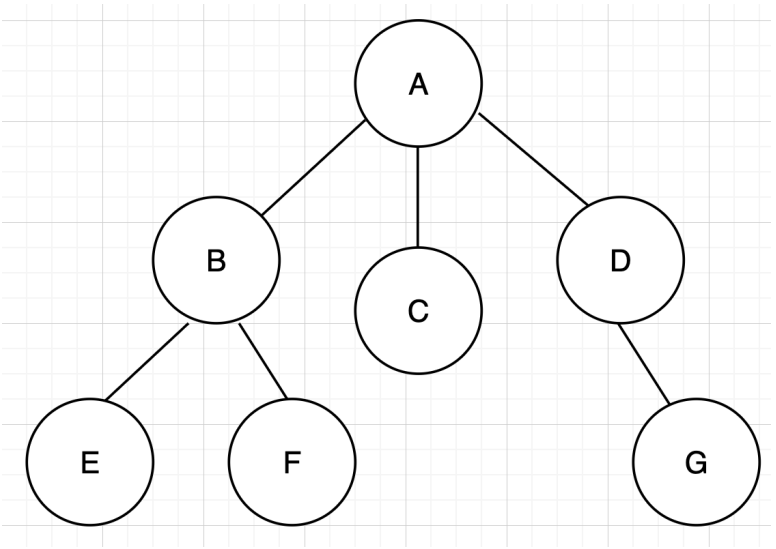
Interestingly, the Tree data structure is organized in a very similar way.

A **Tree** is a non-linear, hierarchical data structure consisting of a collection of **nodes** connected by **edges**.

Unlike arrays or linked lists, data inside a tree is not arranged in a straight sequence. Instead, it is organized in parent-child relationships, creating a branching structure.

A tree always begins with a special node called the **root**, from which all other nodes originate. Each node may have one or more child nodes, and those child nodes can have children of their own, creating a recursive hierarchy.

Consider the following simple tree:



In this example:

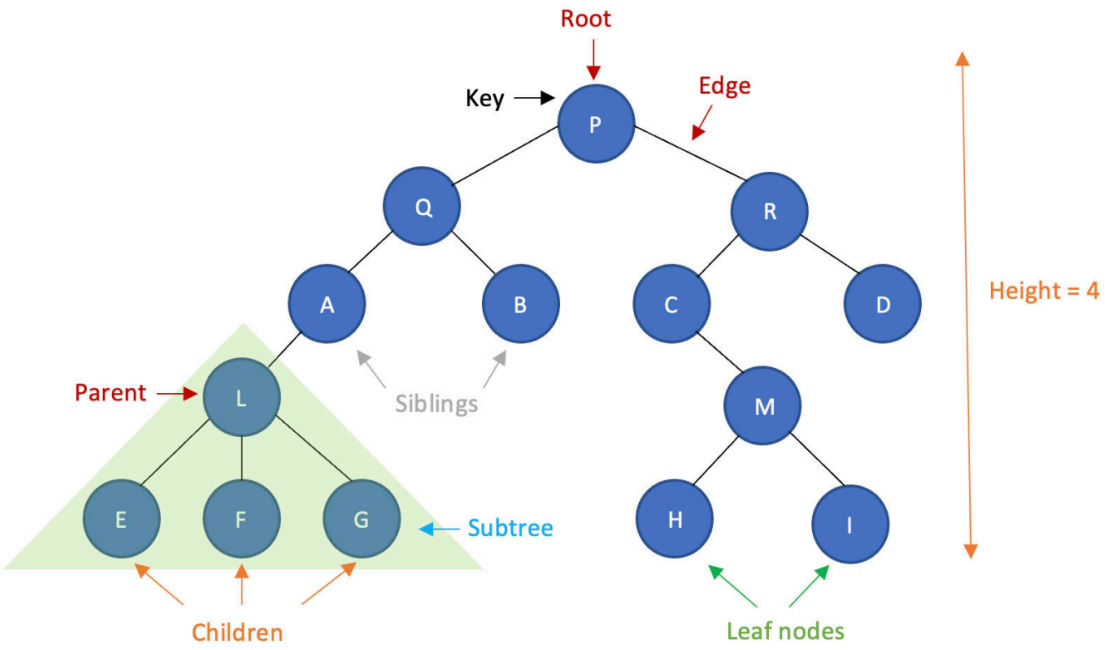
- A is the root node.
- B, C, and D are children of A.
- E and F are children of B.
- G is a child of D.

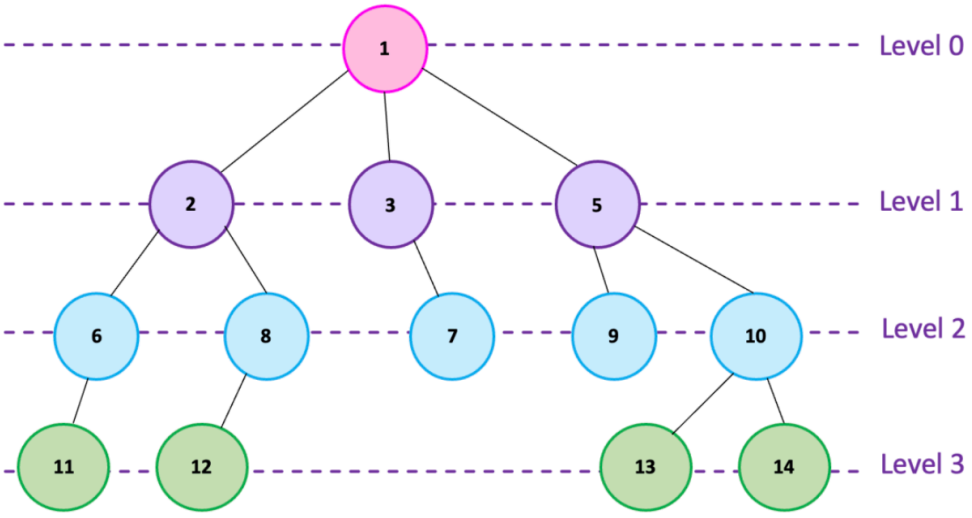
A tree can be viewed as a special type of graph. However, unlike general graphs, a tree **contains no cycles, has exactly one path between any two nodes, has no isolated nodes, remains connected as a single structure.**

These properties make trees efficient for representing hierarchical information.

2. Components of a Tree

To understand trees properly, we must first learn their terminology. The below images are used for reference:





Levels in Tree:

- Node: A **Node** is the basic unit of a tree. Each node stores data and may connect to other nodes.
- Edge: An **Edge** is the connection between two nodes.
- Root: The **Root** is the topmost node of a tree. It has no parent and serves as the starting point of the hierarchy.
- Parent: A **Parent** node is a node that has one or more child nodes.

- Child: A **Child** node is directly connected below a parent node.
- Siblings: Nodes sharing the same parent are called **siblings**.
- Leaf Node: A **Leaf** is a node that has no children. Leaf nodes often represent the end of a branch.
- Internal Node: An **Internal Node** is a node that has at least one child.
- Subtree: A **Subtree** is a smaller tree formed from a node and all of its descendants. Every subtree is itself a valid tree.
- Height: The **Height** of a node is the number of edges in the longest path from that node down to a leaf.
- Depth: The **Depth** of a node is the number of edges from the root to that node.
- Level: The **Level** of a node indicates its position within the tree hierarchy. Nodes with the same depth belong to the same level.
- Path: A **Path** is a sequence of connected nodes.
- Path Length: The **Length of a Path** is **Number of Nodes – 1**
- Degree: The **Degree** of a node is the number of children it possesses.
- Ancestor: If a node lies on the path from the root to another node, it is called an **ancestor**.
- Descendant: A **Descendant** is a node reachable by moving downward from another node.
- Rooted Tree: A **Rooted Tree** is a tree with a designated root node. Most trees studied in computer science are rooted trees because they establish a clear hierarchy and traversal starting point.

Trees come in many different forms. Binary Trees, Binary Search Trees, AVL Trees, Heaps, B-Trees, and many others all originate from the fundamental concepts introduced here. Although they follow different rules, they all share the same basic tree structure and terminology.

3. Applications of Trees

Trees are among the most widely used data structures in computer science because many real-world systems naturally follow a hierarchical structure. Some common applications include:

- Searching and Sorting: Tree-based structures such as Binary Search Trees allow efficient searching, insertion, and deletion operations.
- Building Advanced Tree Structures: Many specialized trees are derived from the basic tree concept, including Binary Search Trees (BST); AVL Trees; B-Trees; Heaps; Syntax Trees; K-D Trees; Suffix Trees; Spanning Trees. Each serves a unique purpose while maintaining the core characteristics of trees.
- File Management Systems: Operating systems organize files and folders using tree structures. Every folder can contain files and subfolders, naturally forming a hierarchy.

```
Documents
├── Reports
│   ├── report1.pdf
│   └── report2.pdf
└── Pictures
    ├── image1.jpg
    └── image2.png
```

- Compilers and Interpreters: Programming languages use trees to represent code structure. For example, arithmetic expressions are often converted into syntax trees before execution.

- Artificial Intelligence: Trees are commonly used in Decision Trees; Expert Systems; Game-playing algorithms; Search algorithms. These applications help machines make intelligent decisions.

- Network Routing and Data Transmission: Communication networks can be represented using trees to determine efficient paths for transmitting information.

Advantages of Trees

- Efficient Operations: Balanced trees such as AVL Trees and Red-Black Trees perform searching, insertion, and deletion in: $O(logn)$, making them significantly faster than many linear structures.
- Natural Hierarchical Representation: Trees mirror many real-world systems naturally, making data easier to organize and understand.
- Recursive Structure: Trees are inherently recursive, making them elegant and powerful when combined with recursive algorithms.
- Flexibility: Trees can grow and shrink dynamically without requiring contiguous memory like arrays.
- Suitable for Large Data: Large datasets become easier to manage because trees organize information efficiently.

Disadvantages of Trees

- Higher Memory Usage: Each node often stores additional references or pointers, increasing memory consumption.
- Complex Implementation: Trees are generally more difficult to implement and understand than arrays or linked lists.
- Performance Depends on Balance: An unbalanced tree can degrade into a structure resembling a linked list. In such cases, $Search = O(n)$ instead of $O(logn)$
- Hash Tables May Be Faster: For pure searching tasks, hash tables can often achieve $O(1)$, which is faster than most tree operations.
- Maintenance Overhead: Advanced trees require balancing algorithms and additional logic to maintain efficiency.

Summary

Trees are one of the most important non-linear data structures in computer science. Unlike linear structures such as arrays, stacks, queues, and linked lists, trees organize data hierarchically, allowing multiple paths and relationships between elements.

A tree consists of nodes connected by edges, beginning with a root node and expanding through parent-child relationships. Understanding concepts such as roots, leaves, ancestors, descendants, depth, height, and subtrees is essential before studying more advanced tree structures.

Trees are used extensively in file systems, databases, search algorithms, compilers, artificial intelligence, and networking. Their ability to represent hierarchical information efficiently makes them one of the most powerful tools in computer science.

As we move forward, we will build upon these fundamentals to study specialized trees such as Binary Trees, Binary Search Trees, AVL Trees, Heaps, and many others that power modern computing systems.

References

https://drive.google.com/file/d/1tOqiD1hwN4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/introduction-to-tree-data-structure/

https://www.geeksforgeeks.org/dsa/tree-data-structure/

https://www.programiz.com/dsa/trees

Next Section:

11.1. Operations on Trees